

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ

КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1

«розгортання середовища та порівняльний аналіз мовних моделей»

з дисципліни: «Обчислювальні методи»

Виконав(-ла):

Студент групи AI-243

Сливка Максим

Олексійович

Перевірів:

ас. Баранов О.М.

Одеса – 2026

Завдання 1 Ціна роздумів: міркувальна vs інструктивна модель

Таблиця 1: Фактичне питання ("Столиця України?")

Показник	qwen3:4b (міркувальна)	llama3.2:3b (інструктивна)
response	Kyiv (Київ)	Київ
eval_count	2169	5
Час, с	111.98	6.85

Таблиця 2: Задача на міркування (Ручка і зошит)

Показник	qwen3:4b (міркувальна)	llama3.2:3b (інструктивна)
Правильних із 5	5	0 (відповідала неправильно)
eval_count	1274	170
Час, с	59.63	4.98

Відповіді чат-бота

1. Куди пішли ці токени?

Ці додаткові токени (понад 2000 у qwen3:4b) пішли на генерацію прихованого блоку роздумів (thinking). Міркувальна модель спочатку "думає вголос" і перевіряє власні припущення у прихованому тексті, і лише після цього видає фінальну коротку відповідь користувачу.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання не виправдане?

Вони дають абсолютну перевагу на математичних, логічних задачах та під час написання коду, де потрібен покроковий аналіз (саме тому Qwen вирішила 5 з 5 задач). Їх використання повністю не виправдане на простих фактичних питаннях (як столиця країни), оскільки це марнотратно спалює час і ресурси системи.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Інструктивні моделі навчають одразу (напрямую) генерувати найбільш імовірну правильну відповідь на запит користувача. Міркувальні ж моделі додатково тренують на спеціальних масивах даних із покроковими розв'язаннями (метод Chain of Thought), щоб змусити їх завжди вибудовувати логічний ланцюжок перед фінальним висновком.

4. Чи означає більша кількість згенерованих токенів у міркувальній моделі, що вона «розумніша» за інструктивну?

Ні, кількість токенів свідчить лише про специфіку її архітектури — "звичку" до внутрішнього діалогу перед будь-якою дією. Розумнішою її робить саме якість цих прихованих роздумів на складних задачах, тоді як на простих питаннях велика кількість токенів просто робить модель дуже повільною.

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b?

Різниця зумовлена фізичним розміром моделей (архітектурою та кількістю ваг). Модель qwen3:4b має 4 мільярди параметрів, тоді як llama3.2:3b — 3 мільярди; тому процесору доводиться виконувати значно більше математичних операцій для обчислення кожного токена Qwen, що знижує швидкість (20 токенів/с проти 49 токенів/с).

Висновок за завданням 1: Міркувальні моделі є чудовим інструментом для задач, що вимагають багатоетапного логічного виведення, оскільки прихований блок thinking допомагає уникати інтуїтивних помилок (як у задачі з ручкою). Проте для простих інтерактивних чат-ботів або фактчекінгу краще використовувати інструктивні моделі, які відповідають швидше і не витрачають зайвих ресурсів на очевидні речі.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3: Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	1877	укр. (з русизмами)	12	Ні (занадто довга)
llama3.2:3b	українською, 2 речення	316	укр. (з англізмами)	2	Так
qwen3:4b	лаконічний асистент	124	чиста українська	2	Так
qwen3:4b	українською, 2 речення	139	чиста українська	1	Так

Таблиця 4: Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	50	588	638
llama3.2:3b	українською, 2 речення	54	82	136
qwen3:4b	лаконічний асистент	64	1654	1718
qwen3:4b	українською, 2 речення	64	691	755

Відповіді на питання

1. **Яка модель точніше дотрималась обмеження «максимум двома реченнями»?**

Модель qwen3:4b дотрималась цього обмеження найкраще, видавши чітку відповідь з 1-2 речень без зайвої "води".

Натомість llama3.2:3b спочатку проігнорувала обмеження, і лише після жорсткої вказівки вклалася у ліміт.

2. **Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?**

Так, локальна інструктивна модель (llama3.2:3b) часто використовувала русизми та англійські слова ("Relation", "tactics"), оскільки їй бракує словникового запасу.

3. **Як змінився prompt_tokens при зміні system-повідомлення і чому?**

Значення prompt_tokens збільшилося (наприклад, з 50 до 54), оскільки нове строге system-повідомлення фізично містить більше символів та слів, які конвертуються у вхідні токени.

4. **Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?**

Оскільки qwen3:4b є міркувальною моделлю, вона витрачає сотні токенів на генерацію прихованого масиву роздумів (thinking), і лише потім формує видиму коротку відповідь.

Відповіді чат-бота

1. **Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?**

Це залежить від архітектури моделі та обсягу її навчальних даних.

Міркувальна модель спочатку аналізує інструкцію у прихованому блоці, що допомагає їй краще виконати обмеження. А якість мови прямо залежить від відсотка україномовних текстів у базі тренування конкретної моделі.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

System-повідомлення відіграє роль "базових налаштувань" і задає загальний тон, стиль та обмеження на весь діалог. Під час навчання моделям "прищеплюють" сприймати system-повідомлення як непорушне правило від розробника, тому зазвичай воно має вищий пріоритет, ніж плинні запити користувача.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Міркувальна модель запрограмована завжди будувати прихований логічний ланцюжок (Chain of Thought), перш ніж згенерувати фінальний текст. Ці "невидимі" токени процесор обчислює фізично так само, як і звичайні, що призводить до різкого зростання показника completion_tokens.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Частіше за все сучасні моделі нададуть перевагу інструкції з user-повідомлення, оскільки воно знаходиться ближче до кінця контексту і сприймається як "найсвіжіший" запит. Проте результат є непередбачуваним: модель може заплутатися, спробувати об'єднати обидві вказівки або зламати формат відповіді.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Малі моделі мають жорсткий ліміт "мозку" (кількості параметрів), тому їх навчають переважно на англійських даних для економії ресурсів. Через критичний дефіцит українських текстів у їхніх вагах вони погано розуміють граматику і легко "зриваються" на суржик або англійську, коли не знають точного перекладу.

Висновок до завдання 2: System-повідомлення є зручним базовим інструментом для керування поведінкою моделі, проте воно не гарантує 100% надійності. Експеримент показав, що малі мовні моделі часто ігнорують прямі текстові інструкції, особливо щодо суворих обмежень формату та обсягу. Самого лише промпу недостатньо для стабільної роботи систем; якщо модель ігнорує правила, варто переходити на використання міркувальних моделей, надавати їй чіткі приклади (few-shot prompting) або програмно обмежувати формат її виводу.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна (llama3.2:3b)	Хмарна (qwen/qwen3.6-27b)
Час відповіді, с	9.52 с	4.66 с
completion_tokens	169	552 (з роздумами)
Довжина відповіді, символів	~430	~440 (видимого тексту)
Мова відповіді	укр. (з суржилом)	ідеальна українська
Фактичних помилок	Так (помилка контексту)	Немає

Відповіді на питання

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Довелося змінити лише конфігураційні параметри: `base_url`, `api_key` та назву `model` (фактично 3 рядки в словнику `providers.py`). Сама логіка надсилання запиту (`client.chat.completions.create`) залишилася незмінною завдяки сумісності стандартів.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь хмарної моделі була значно повнішою та логічнішою. Локальна модель плутала контекст, генеруючи набір загальних фраз про математику, тоді як хмарна чітко структурувала текст і розкрила відмінності між функцією та методом саме в програмуванні.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Так, локальна модель мала величезні проблеми з мовою: вона використовувала суржик, русизми (наприклад, «tactics», «comportуючись») та періодично втрачала логічний зв'язок між реченнями. Хмарна ж модель видала ідеально грамотний український текст.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Хмарна модель працює на потужних промислових кластерах зі спеціалізованими прискорювачами (наприклад, чіпах LPU від Groq або надшвидких H100). Їхня колосальна обчислювальна потужність з надлишком компенсує затримки на передачу даних мережею.

Відповіді чат-бота

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Компанія OpenAI першою створила масовий і зручний API, навколо якого миттєво виросла екосистема бібліотек та інструментів. Іншим провайдерам (як і розробникам Ollama) стало стратегічно вигідно підтримувати цей формат, щоб розробники могли переходити на їхні сервіси без переписування жодного рядка коду.

2. Наскільки розмір моделі (3 млрд проти хмарної 20-70 млрд) впливає на якість відповіді порівняно з іншими факторами?

Розмір моделі (кількість параметрів) є найбільш критичним фактором. Хоча якість навчальних даних важлива, мала модель фізично не має достатньої ємності "мозку", щоб запам'ятати глибокі нюанси граматики неанглійських мов (української) та утримувати складні логічні ланцюжки.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Провайдери хмарних API використовують апаратне забезпечення, яке спеціально спроектоване для інференсу нейромереж. Домашній комп'ютер (навіть із потужною ігровою відеокартою) у сотні разів поступається промисловій архітектурі в пропускній здатності пам'яті.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Головний ризик — передача конфіденційних даних клієнтів або пропрієтарного коду на чужі сервери. У комерції ці ризики мінімізують шляхом анонімізації даних перед відправкою, укладання жорстких корпоративних угод (NDA/BAA) з хмарними провайдерами, або використання локальних моделей у власному контурі.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit — це штучне обмеження від провайдера на кількість запитів або токенів за хвилину (як ми стикалися з 429 помилкою на Groq). Розробники обходять це, прописуючи в коді алгоритми паузи та повторних спроб (exponential backoff), чергуванням кількох безкоштовних API-ключів, або просто купуючи платний тариф.

Висновок до завдання 3: Локальну модель (навіть попри її слабкість) я б обрав виключно для завдань, що вимагають 100% приватності (аналіз медичних, фінансових даних чи закритого корпоративного коду), або коли немає доступу до інтернету. У всіх інших випадках хмарна модель є безальтернативним вибором, оскільки вона забезпечує миттєву генерацію, бездоганну українську мову, здатність до складної аналітики та стабільну роботу з форматами даних.

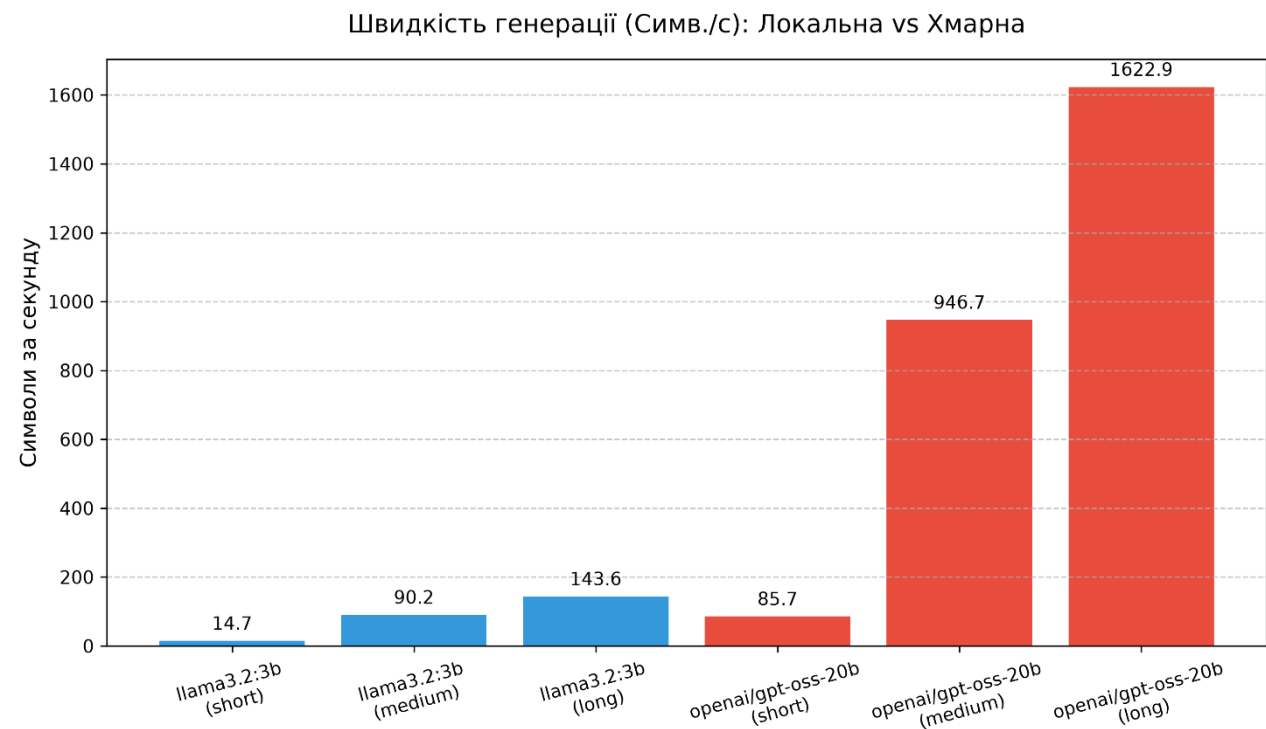
Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.079	2.377	14.7
локальна	llama3.2:3b	середній	2.086	3.914	90.2
локальна	llama3.2:3b	довгий	2.101	12.540	143.6
хмарна	openai/gpt-oss-20b	короткий	0.446	0.478	85.7
хмарна	openai/gpt-oss-20b	середній	0.332	0.505	946.7
хмарна	openai/gpt-oss-20b	довгий	0.480	1.375	1622.9

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	8.52	2.08
Всього, с	8.81	2.37



Відповіді на питання

1. У якої моделі кращий TTFT і чому саме в неї?

Кращий TTFT продемонструвала хмарна модель (0.3–0.5 с проти 2.1 с у локальній). Вона працює на спеціалізованому промисловому обладнанні, яке обробляє вхідний контекст майже миттєво, з лишком перекриваючи навіть затримку мережі.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Так, у нашому експерименті ці показники повністю збігаються. Хмарна модель виграла і за швидкістю реакції (до першого символу), і за загальною швидкістю генерації тексту.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпу до довгого?

Розрив у швидкості генерації (симв/с) колосально зріс на користь хмарної моделі. На короткому промпті хмарна модель була швидшою приблизно у 6 разів, а на довгому промпті розрив збільшився вже в 11 разів.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Значення дещо коливалися через зміну завантаженості локального процесора та стрибки мережевого пінгу. Одного запуску категорично недостатньо, адже він може потрапити на випадковий сплеск навантаження (тому ми брали медіану).

5. Чому холодний старт повільніший за теплий?

Під час холодного старту масивний файл ваг моделі (кілька гігабайтів) фізично завантажується з жорсткого диска/SSD в оперативну пам'ять комп'ютера, що потребує багато часу. При теплому старті модель уже знаходиться в пам'яті і готова до миттєвих обчислень.

Відповіді чат-бота

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Під час TTFT модель виконує початкове паралельне обчислення (префіксне кешування та attention) над усім текстом вхідного промпу, не генеруючи нових слів. Цей показник вимірюють окремо, бо саме він найбільше впливає на психологічне відчуття "чуйності" системи для користувача, тоді як загальна швидкість показує ефективність масової видачі тексту.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Хмарна модель може мати гірший TTFT виключно через мережеві затримки (пінг до віддаленого сервера може складати кілька сотень мілісекунд). Однак після подолання цієї затримки потужні серверні GPU генерують кожен наступний токен у десятки разів швидше за відеокарту локального комп'ютера.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт — це процес ініціалізації, коли система виділяє відеопам'ять та копіює мільярди параметрів моделі зі сховища (SSD) у VRAM. У наступних (теплих) запусках ваги вже завантажені і знаходяться у швидкій пам'яті, тому генерація починається без етапу зчитування диска.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Обчислювальні системи не працюють в ідеальному вакуумі: фонові процеси ОС, стрибки затримки інтернету або тимчасове навантаження на сервери можуть легко спотворити один вимір. Медіана ефективно відкидає випадкові аномальні затримки і відображає найбільш типовий, реальний результат.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Вхідний запит (промпт) обробляється паралельно: матричні множення для всіх токенів запиту виконуються одночасно завдяки специфіці архітектури Transformer. Генерація ж відбувається авторегресійно (суворо послідовно) — щоб згенерувати нове слово, модель повинна дочекатися завершення обчислень для всіх попередніх токенів.

Висновок до завдання 4: Для інтерактивного чат-бота критично важливим є показник TTFT (Time To First Token), оскільки користувач психологічно не любить чекати більше 1–2 секунд на початок відповіді — швидкий старт створює ілюзію "живого" спілкування. Натомість для фонові обробки великих обсягів тексту (наприклад, саммаризації книг або автоматичного аналізу логів) TTFT взагалі не має значення. У фонових задачах головним є показник Символів за секунду (або Токенів/с), адже саме він визначає загальну пропускну здатність системи (throughput) і впливає на те, скільки часу піде на обробку гігабайтів інформації.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 13: Оцінка якості за рубрикою

Завдання	llama3.2:3b	Хмарна (openai/gpt-oss-20b)	qwen3:4b
1. Факт	1	2	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	2	1	2
5. Логіка	0	2	2
6. Українська	0	2	2
Разом /12	5 / 12	11 / 12	12 / 12

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	Хмарна (openai/gpt-oss-20b)	qwen3:4b
1. Факт	так	ні	Ні
2. Формат	так	ні	Ні
3. Стиснення	так	так	Так
4. Код	так	так	Ні
5. Логіка	так	так	так
6. Українська	так	так	так

Відповіді на питання

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів набрала локальна міркувальна модель qwen3:4b (12/12) та хмарна openai/gpt-oss-20b (11/12), тоді як інструктивна llama3.2:3b провалилася (5/12). Основна різниця виникла в математичній логіці (завдання 5), збереженні чисел під час стиснення (завдання 3) та якості генерації українського тексту (завдання 6), де інструктивна локальна модель не впоралась.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Оцінки двох прогонів найбільше розійшлися у творчих завданнях (мова, логіка, генерація коду) та у дотриманні формату без markdown. Це свідчить про недетермінованість моделей: їхня надійність не є 100-відсотковою, і результат (особливо оформлення коду чи вибір мови) може випадково змінюватися при повторному запуску.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так, усі три моделі (зокрема qwen3:4b та llama3.2:3b) успішно виконали цю вимогу в завданні 2, згенерувавши чистий JSON. Проте, звичка моделей до

markdown виявилася дуже сильною: у завданні 4 на генерацію коду хмарна модель та llama3.2:3b усе одно обгорнули код у markdown-блоки, проігнорували схожу інструкцію. Лише qwen3:4b змогла видати чистий код.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Хмарна модель та локальна qwen3:4b впоралися ідеально, видавши чистий український текст. А от локальна інструктивна llama3.2:3b зазнала повного провалу: вона генерувала суржик, вставляла англійські слова ("own gravity"), польські ("jej", "Збагненоść") та навіть арабську в'язь замість українських літер.

Приклад провалу

Ось найяскравіший приклад провалу локальної моделі (llama3.2:3b) із мого JSON-файлу:

- Повний текст запиту: "Зонд летить зі швидкістю 60000 км/год, відстань 480 млн км. Скільки повних діб триватиме політ? Покажи хід розв'язання."
- Повний текст відповіді (без скорочень, уривок для звіту): "1. Зробіть оберотний діл на відстань 480 млн км: 480000000000 м/с 2. Подіть відстань на 2 частини 480000000000 м/с: 480000000000 м/с / 2 = 240000000000 м/с 3. Потім роздільте ці дві частини на 2 частини: 240000000000 м/с / 2 = 120000000000 м/с ... 12. Розв'яжайте формулу: 1 доба = 29,28 доби ... 26. Діліть повну добу на 24 доби: 0,000086 доби / 24 доби = 0,00000357 доби"
- Яка правильна відповідь мала бути: 333 повні доби (480 000 000 / 60 000 = 8000 годин; 8000 / 24 = 333.3).
- Пояснення, чому так сталося: Локальна модель не має вбудованого калькулятора чи математичного модуля. Вона просто генерує слова, які статистично часто стоять поруч у навчальних текстах з математики. Заблукавши у великих числах на першому ж кроці, модель увійшла в безглуздий цикл нескінченного ділення на 2, намагаючись імітувати "хід розв'язання", але повністю втративши будь-який логічний сенс.

Відповіді чат-бота

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

На практиці широко використовують автоматизовані бенчмарки (наприклад, MMLU для перевірки знань, HumanEval для коду). Також дуже популярним є підхід "LLM-as-a-Judge", коли потужніша модель

автоматично оцінює відповіді менших моделей, та "сліпе" A/B тестування на реальних користувачах (наприклад, Chatbot Arena).

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

У базах даних, на яких навчали моделі (наприклад, StackOverflow чи GitHub), майже весь програмний код та JSON-структури оформлені у markdown-блоки. Ця статистична "звичка" у вагах моделі виявляється настільки сильною, що вона підсвідомо переборює пряму текстову вказівку користувача.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Навчальний датасет малих моделей на 90% складається з англійських текстів. Через жорстке обмеження кількості параметрів ("мозку" моделі) розробники свідомо жертвують багатомовністю, тому модель просто не здатна вмістити глибоке розуміння граматики та широкого словникового запасу української мови.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Усі сучасні мовні моделі генерують текст імовірносно. Під час повторного запуску (якщо параметр `temperature > 0`) модель може випадково обрати інший синонім на початку речення, що поведе всю подальшу структуру тексту та хід думок зовсім іншим шляхом.

5. Що таке галюцинація мовної моделі і чому модель радше вигадас факт, ніж визнає незнання?

Галюцинація — це генерація моделлю правдоподібного, але хибного або вигаданого факту. Архітектура моделі змушує її завжди намагатися продовжити текст найбільш статистично ймовірними словами; якщо справжніх знань у її вагах немає, вона просто конструює "найбільш схожу на правду" відповідь замість рефлексії власного незнання.

Висновок до завдання 5: Оцінка за рубрикою виявилася надзвичайно надійною, адже вона дозволила замінити суб'єктивне враження ("ця модель здалася мені розумнішою") на конкретні об'єктивні числа (11/12 балів проти 5/12). Найпоказовішими для порівняння стали завдання на логіку та генерацію тексту українською мовою, де локальна модель зазнала катастрофічного провалу. Спираючись на ці дані, для будь-якої серйозної роботи з українськомовним текстом я б безперечно обрав хмарну модель, оскільки вона гарантує граматичну правильність, глибоке розуміння логіки та відсутність непередбачуваних переходів на інші мови чи суржик.

Завдання 6. Керування випадковістю: `temperature` і `top_p`

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	2	5
temperature	0.7	5	—
temperature	1.5	5	1
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 17: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на питання

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Я встановлю `temperature = 0.0`. Це гарантує максимальну детермінованість і мінімізує ризик того, що модель випадково згенерує зайві символи чи пояснення, які зламують структуру JSON.

2. Яке — генератору варіантів заголовків?

Для генератора заголовків оптимальним буде `temperature = 0.7`. Це дозволить отримувати креативні та різноманітні варіанти, зберігаючи при цьому їх логічність та адекватність.

3. Чим вплив `top_p` відрізняється від впливу `temperature` за характером, а не за силою?

`Temperature` математично перерозподіляє ймовірності між усіма токенами (робить їх більш рівномірними або "гострими"), тоді як `top_p` буквально відрізає "хвіст" найменш імовірних слів, забороняючи моделі їх використовувати взагалі.

4. Чи означає `temperature=0` повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Мій експеримент показав 100% збіг (10 із 10 відповідей були однаковими), тому зазвичай це дає високу відтворюваність. Проте технічно повна абсолютна відтворюваність не гарантується через специфіку оптимізації операцій з плаваючою комою на відеокартах.

5. Наведіть найбезглуздішу відповідь, яку модель видала при `temperature=1.5`, і поясніть, чому вона взагалі могла з'явитися.

Найбезглуздішою була відповідь про те, що перший iPhone випустив "Google у Лос-Анджелесі" із вставками арабських слів ("с 2007 سنة"). Це сталося тому, що висока температура "згладила" ймовірності, дозволивши моделі обирати випадкові, статистично малоімовірні слова замість очевидних.

Відповіді чат-бота

1. Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена?

Математично значення `temperature` ділить сирі бали моделі (логіти) перед застосуванням функції `softmax`. Якщо `temperature < 1`, розподіл стає "гострішим" (модель стає впевненішою в лідерах), а якщо `temperature > 1`, розподіл згладжується, роблячи ймовірності всіх доступних токенів більш однаковими.

2. Чим `top_p` відрізняється від `temperature` — обидва впливають на різноманітність, але за яким механізмом?

`Top_p` (nucleus sampling) сортує токени за ймовірністю і відкидає ті, що опиняються за межами заданого відсотка сумарної ймовірності (наприклад, залишає лише топ-90%). Тобто `temperature` змінює самі цифри ймовірностей для всього словника, а `top_p` працює як жорсткий фільтр, що фізично обмежує вибір.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Офіційна документація більшості провайдерів (включаючи OpenAI) рекомендує змінювати лише один із цих параметрів. Оскільки обидва інструменти впливають на один і той самий процес вибору наступного слова, їх одночасна зміна робить поведінку моделі вкрай непередбачуваною.

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Навіть при `temperature = 0` (greedy decoding) архітектура сучасних графічних процесорів оптимізує паралельні матричні обчислення. Через це операції з плаваючою комою можуть мати мікроскопічні відхилення при різних запусках, що іноді призводить до обрання іншого токена на межі ймовірностей.

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для екстракції структурованих даних (JSON, XML, SQL-запити) або написання коду завжди використовують `temperature = 0.0`, щоб унеможливити галюцинації у синтаксисі. Для креативного копірайтингу, перекладів чи брейншторму ідей оптимальними вважаються значення від 0.7 до 0.9.

Висновок до завдання 6: У наступних лабораторних роботах, де від агентів вимагатиметься повертати строго структуровані дані (наприклад, виклики інструментів у JSON-форматі), я безперечно використовуватиму temperature = 0.0. Налаштування "за замовчуванням" (яке зазвичай складає близько 0.7 в API багатьох провайдерів) націлене на креативність та імітацію живої розмови. Якщо залишити його без змін для агентських завдань, модель рано чи пізно згенерує випадковий непередбачений символ або змінить ключі формату, що гарантовано призведе до падіння парсера коду та збою всієї програми.

ЗАГАЛЬНИЙ ВИСНОВОК

Таблиця 18: Підсумкове порівняння моделей

Критерій	Локальна	Хмарна
Швидкість генерації	Повільна (14-143 симв/с)	Надшвидка (85-1622 симв/с)
TTFT	Високий (~2-9 с)	Низький (~0.3-0.5 с)
Якість, балів /12	5 / 12	11 / 12
Стабільність формату	Добра (видає сирий JSON)	Середня (додає markdown)
Потрібні ресурси	Високі (8+ ГБ RAM/VRAM)	Мінімальні (лише браузер/інтернет)
Вартість	Безкоштовно	Безкоштовний ліміт / Платно
Куди йдуть ваші дані	Залишаються на ПК	Відправляються на сервери провайдера
Працює без інтернету	Так	Ні
Коли обирати саме її	Локальну модель слід обирати виключно тоді, коли критично важливою є повна конфіденційність та безпека даних (аналіз медичних карток, закритий корпоративний код, банківська таємниця), або коли система має працювати автономно без доступу до інтернету. У цих випадках ми свідомо йдемо на компроміс, приймаючи її	Хмарну модель слід обирати для всіх інших production-завдань, де потрібна висока якість тексту (бездоганна українська мова), здатність розв'язувати складні логічні/математичні задачі, стабільно генерувати код та JSON, а також забезпечувати миттєву швидкість (низький TTFT). Обираючи її, ми свідомо погоджуємося на залежність від інтернет-з'єднання, лімітів провайдера та передачу даних назовні.

	повільну швидкість, суржик та нездатність до складної логіки, заради повного контролю над даними.	
--	---	--

Під час виконання лабораторної роботи було успішно розгорнуто локальне та хмарне середовища для роботи з великими мовними моделями (LLM). Проведено комплексний порівняльний аналіз моделей різної архітектури (міркувальні та інструктивні) і різного масштабу (локальні на 3-4 млрд параметрів та хмарні на 20+ млрд параметрів).

Спираючись на отримані експериментальні дані, можна зробити наступні ключові висновки:

- Міркувальні vs Інструктивні моделі:** Міркувальні моделі (наприкладі qwen3:4b) генерують прихований ланцюжок думок (Chain of Thought), що робить їх значно повільнішими і «дорожчими» за кількістю вихідних токенів, але дає абсолютну перевагу у вирішенні логічних та математичних задач. Використовувати їх для простих фактичних питань — недоцільно.
- Керування поведінкою (System prompt):** System-повідомлення задає базовий формат, проте малі локальні моделі часто ігнорують прямі обмеження (обсяг, формат або заборону markdown). Для забезпечення машинної зчитуваності відповідей (наприклад, JSON) самої лише текстової інструкції недостатньо.
- Швидкість та TTFT:** Хмарні моделі, попри затримки на передачу даних мережею, значно перевершують локальні як за часом до першого токена (TTFT), так і за загальною пропускну здатністю (символів/с). Це зумовлено використанням потужних промислових прискорювачів.
- Оцінка якості та галюцинації:** Оцінювання за фіксованою рубрикою довело беззаперечну перевагу хмарної моделі. Локальна модель (Llama3.2:3b) продемонструвала схильність до галюцинацій у складних розрахунках та катастрофічний брак знань української граматики (генерація суржику та слів з інших мов).
- Параметри генерації:** Температура (temperature) є критичним параметром контролю. Для інтеграції моделей у програмний код (отримання JSON) необхідно завжди використовувати temperature = 0.0 для забезпечення детермінованості, тоді як для творчих завдань оптимальними є показники 0.7–1.0.

Головний підсумок: У сучасній інженерії не існує універсально «найкращої» моделі. Вибір завжди є компромісом: локальні моделі є незамінними для

роботи в ізольованих середовищах та збереження абсолютної приватності корпоративних даних, тоді як хмарні API є стандартом для побудови швидких, надійних та інтелектуально складних production-систем, орієнтованих на кінцевого користувача.